

B4:

What should be the return type of this method?

--> Trả lời:

Kiểu trả về của phương thức toString() trong Java luôn phải là String.

Phương thức này được override từ lớp Object, nhằm cung cấp một biểu diễn dạng chuỗi của đối tượng.

B13:

Which classes are aggregates of other classes?

--> Trả lời:

CompactDisc là một aggregate của Track: một đĩa CD bao gồm một tập hợp các đối tượng Track.

Cart là một aggregate của Media: giỏ hàng chứa danh sách các sản phẩm Media.

Store là một aggregate của Media: cửa hàng quản lý một tập hợp các sản phẩm Media.

B15:

If the passing object is not an instance of Media, what happens?

--> Trả lời:

Khi có kiểm tra instanceof: phương thức sẽ trả về false nếu đối tượng không cùng kiểu. Điều này giúp chương trình an toàn hơn, tránh lỗi NullPointerException (khi đối tượng là null) và đảm bảo tính hợp lý (hai đối tượng khác kiểu thì không thể bằng nhau).

Nếu bỏ qua instanceof và ép kiểu trực tiếp (casting): chương trình có thể ném ra ClassCastException, dẫn đến lỗi runtime và làm gián đoạn thực thi.

Kết luận: Việc kiểm tra kiểu dữ liệu là cần thiết để đảm bảo độ ổn định của chương trình và tuân thủ các nguyên tắc thiết kế hướng đối tượng trong Java.

B16:

1. What are the advantages of Polymorphism?

--> Trả lời:

Khả năng mở rộng (Extensibility): Bạn có thể bổ sung các lớp con mới (ví dụ Magazines) mà không cần chỉnh sửa phần code lập để in thông tin; hệ thống vẫn tự xử lý được.

Tái sử dụng mã nguồn (Reusability): Chỉ cần viết logic một lần cho lớp cha (Media), và nó sẽ áp dụng đúng cho tất cả các lớp con.

Giảm phụ thuộc (Loose Coupling): Tách biệt giữa việc gọi phương thức và cách nó được triển khai, giúp hệ thống linh hoạt và dễ bảo trì hơn.

Đơn giản hóa code: Thay vì dùng nhiều if-else hay switch-case để kiểm tra kiểu đối tượng, bạn chỉ cần gọi một dòng duy nhất m.toString().

2. How is Inheritance useful to achieve Polymorphism in Java?

--> Trả lời:

Thiết lập quan hệ “Is-A”: Kế thừa tạo ra cấu trúc phân cấp. Để một List<Media> có thể chứa cả CD và DVD, thì các lớp này bắt buộc phải kế thừa từ Media.

Cơ chế ghi đè (Overriding): Nhờ kế thừa, lớp con có thể định nghĩa lại hành vi của lớp cha. Nếu không có kế thừa, việc override phương thức sẽ không thực hiện được, và đa hình cũng không thể xảy ra.

Ép kiểu lên (Upcasting): Kế thừa cho phép xem một đối tượng lớp con như đối tượng của lớp cha (ví dụ Media m = new Book()), đây chính là nền tảng để triển khai đa hình trong Java.

3. What are the differences between Polymorphism and Inheritance in Java? (Sự khác biệt giữa Đa hình và Kế thừa?)

--> Trả lời:

Về bản chất và định nghĩa:

Kế thừa: Là cơ chế cho phép lớp con kế thừa thuộc tính và phương thức từ lớp cha, tạo quan hệ “IS-A” (ví dụ: CompactDisc là một Media).

Đa hình: Là khả năng các đối tượng khác lớp phản hồi cùng một lời gọi phương thức theo những cách khác nhau (ví dụ cùng gọi toString() nhưng mỗi lớp trả về nội dung riêng).

Về mục đích:

Kế thừa: Hướng đến tái sử dụng mã nguồn, giúp tránh lặp lại code.

Đa hình: Tập trung vào tính linh hoạt và khả năng mở rộng, cho phép hệ thống dễ dàng thích nghi với các lớp mới.

Về thời điểm xác định:

Kế thừa: Được thiết lập khi biên dịch, thông qua từ khóa extends.

Đa hình: Thể hiện rõ ở runtime (đa hình động), khi Java quyết định phương thức nào được gọi dựa trên đối tượng thực tế.

Về mối quan hệ:

Kế thừa là nền tảng để tạo ra đa hình.

Đa hình là kết quả khi kết hợp kế thừa với cơ chế ghi đè phương thức (@Override).

Về khái niệm và từ khóa liên quan:

Kế thừa: extends, super, lớp cha (base class), lớp con (derived class).

Đa hình: Override, Overload, Dynamic Binding, Upcasting.

B17:

1: What class should implement the Comparable interface?

--> Trả lời:

Lớp Media nên implement interface Comparable<Media>.

Giải thích: Vì Cart quản lý một danh sách các đối tượng kiểu Media, nên để sử dụng Collections.sort(itemsOrdered) mà không cần truyền thêm comparator, mỗi đối tượng trong danh sách (tức Media và các lớp con) phải tự định nghĩa được cách so sánh với nhau thông qua phương thức compareTo().

2: In those classes, how should you implement the compareTo() method to reflect the ordering that

we want?

--> Trả lời:

Bước 1: So sánh tiêu chí chính (Title)

Dùng compareToIgnoreCase() của String để so sánh this.title với other.title, giúp sắp xếp theo thứ tự alphabet mà không phân biệt chữ hoa – thường.

Bước 2: Xử lý kết quả so sánh tiêu đề

Nếu kết quả  $\neq 0$ : trả về luôn để xác định thứ tự.

Nếu kết quả = 0: hai đối tượng trùng tiêu đề  $\rightarrow$  chuyển sang bước tiếp theo.

Bước 3: So sánh tiêu chí phụ (Cost)

Dùng `Float.compare()` để so sánh giá. Do yêu cầu “giá cao hơn đứng trước” (giảm dần), nên đảo thứ tự: `Float.compare(other.cost, this.cost)`.

Bước 4: Trả về kết quả

Phương thức `compareTo()` trả về một số nguyên:

Số âm  $\rightarrow$  đối tượng hiện tại đứng trước

Số dương  $\rightarrow$  đối tượng hiện tại đứng sau

0  $\rightarrow$  hai đối tượng ngang nhau trong thứ tự sắp xếp

3: Can we have two ordering rules of the item (by title then cost and by cost then title) if we use this Comparable interface approach?

--> Trả lời --> Không.

Giải thích: Interface Comparable chỉ cho phép định nghĩa duy nhất một phương thức `compareTo()`, tương ứng với một “thứ tự sắp xếp tự nhiên” (natural ordering) của đối tượng.

Nếu bạn muốn hỗ trợ nhiều cách sắp xếp khác nhau, chẳng hạn như “sắp xếp theo tên” hoặc “sắp xếp theo giá” thông qua lựa chọn của người dùng, thì Comparable là không đủ. Khi đó, bạn cần sử dụng Comparator để định nghĩa nhiều tiêu chí so sánh linh hoạt khác nhau.

4: Suppose the DVDs have a different ordering rule from the other media types, that is by title, then decreasing length, then cost. How would you modify your code to allow this?

--> Trả lời:

#### Ý 1: Ưu tiên so sánh thuộc tính chung (Title)

Bắt đầu bằng việc so sánh title của hai đối tượng Media. Vì mọi loại Media (Book, CD, DVD) đều có tiêu đề, đây là tiêu chí đầu tiên. Nếu tiêu đề khác nhau, trả về kết quả ngay.

#### Ý 2: Kiểm tra kiểu đối tượng (Type Checking)

Nếu tiêu đề giống nhau, kiểm tra xem cả this và other có cùng là DigitalVideoDisc bằng toán tử instanceof. Điều này cho phép xử lý riêng cho DVD mà không ảnh hưởng đến các loại khác.

#### Ý 3: Ép kiểu và so sánh thuộc tính đặc thù (Length)

Khi đã xác nhận cả hai đều là DVD, tiến hành ép kiểu về DigitalVideoDisc để truy cập thuộc tính length. So sánh độ dài theo thứ tự giảm dần (đĩa dài hơn đứng trước). Nếu khác nhau thì trả về luôn kết quả.

#### Ý 4: So sánh tiêu chí cuối (Cost)

Nếu hai DVD có cùng tiêu đề và độ dài, hoặc nếu không phải DVD (ví dụ hai Book trùng tên), thì dùng cost để phân định. So sánh giá theo thứ tự giảm dần để hoàn tất việc sắp xếp.